

Department of Computer Science and Engineering

DSA01 — Object Oriented Programming with C++

Smart Hospital Bed & Patient Monitoring Management System

Assignment Submission

Course Outcomes (CO)	CO1, CO2, CO3
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping	SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure

Name	Register Number
Mohammed Farmanullah R	1972260040
B. Balasuriya	1911260010
Bala Ashwin R	1921260102

1. Problem Statement

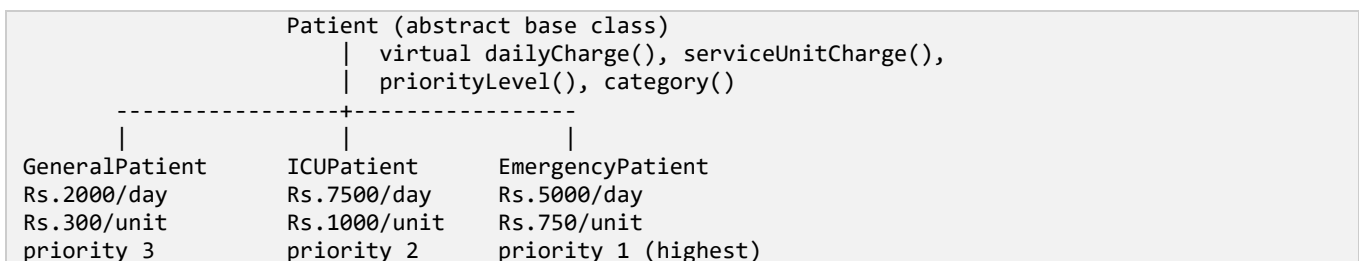
A medium-sized hospital has a limited number of beds, multiple patient categories, and manual monitoring, which makes bed allocation and service tracking difficult. This assignment designs, implements, and evaluates a menu-driven, object-oriented C++ application — the Smart Hospital Bed & Patient Monitoring Management System — that automates patient registration, bed allocation, admission, daily monitoring updates, billing, discharge, admission comparison, and bed-utilization reporting.

The core entities — Patient, HospitalBed, and Admission — are modelled as encapsulated classes. Patient categories (GeneralPatient, ICUPatient, EmergencyPatient) form an inheritance hierarchy that uses virtual functions so that daily charges, monitoring/service charges, and priority behaviour adapt at run time. Robustness is ensured through default, parameterized, and copy constructors; destructors that release beds and finalize records; and operator overloads for comparing admissions and formatting report output.

2. Class Design Overview

Class	Responsibility	Key OOP Feature(s)
Patient (abstract base)	Common ID, name, age, diagnosis; pure virtual billing/priority behaviour	Encapsulation, abstraction
GeneralPatient / ICUPatient / EmergencyPatient	Category-specific daily charge, service charge, priority level	Inheritance, method overriding, polymorphism
HospitalBed	Bed number, ward type, occupancy status, occupant patient ID	Encapsulation
Admission	Links a patient to a bed; tracks days, service units, bill, active status	Constructors, destructor, operator overloading
Hospital	Owns collections of patients, beds and admissions; allocation logic, reports, menu	Aggregation, control logic, static-member coordination

Inheritance Hierarchy



Billing Rules Used

Patient Type	Base Daily Bed Charge	Monitoring/Service Charge
General Patient	Rs. 2,000 / day	Rs. 300 per unit
ICU Patient	Rs. 7,500 / day	Rs. 1,000 per unit
Emergency Patient	Rs. 5,000 / day	Rs. 750 per unit

3. Pseudocode

```
BEGIN HospitalSystem

  DEFINE class Patient (abstract base)
    fields: patientID, name, age, diagnosis
    static field: totalPatients
    abstract methods: dailyCharge(), serviceUnitCharge(), priorityLevel(), category()

  DEFINE class GeneralPatient, ICUPatient, EmergencyPatient EXTENDS Patient
    override dailyCharge(), serviceUnitCharge(), priorityLevel(), category()
    // General: Rs.2000/day, Rs.300/unit, priority 3
    // ICU:      Rs.7500/day, Rs.1000/unit, priority 2
    // Emergency: Rs.5000/day, Rs.750/unit, priority 1 (highest)

  DEFINE class HospitalBed
    fields: bedNumber, wardType, occupied, occupantID
    methods: occupy(id), vacate(), display()

  DEFINE class Admission
    fields: admissionID, patientID, bedNumber, days, serviceUnits, bill, active
    static fields: totalAdmissions, totalRevenue
    methods: updateService(units), calculateBill(), discharge()
    operator> : compare two admissions by bill
    operator<< : format admission record as report line

  DEFINE class Hospital
    fields: patients[], beds[], admissions[]
    methods:
      registerPatient()      -> create derived Patient object by category, store
      displayPatients()      -> loop, call virtual display() on each (polymorphism)
      displayBeds()          -> loop beds, print status
      admitPatient()         -> find patient, find free bed matching ward/priority,
                             mark bed occupied, create Admission record
      updateMonitoring()     -> find active admission, add service units
      calculateBill()        -> find active admission, set days, compute bill
      dischargePatient()     -> find admission, finalize bill, vacate bed, mark inactive
      compareAdmissions()    -> fetch two admissions, use operator> to compare bills
      utilizationReport()    -> count occupied/available beds, compute %, show totals

  MAIN:
    CREATE Hospital with N fixed beds
    LOOP until user selects Exit:
      PRINT menu (10 options)
      READ choice
      VALIDATE choice (re-prompt on invalid input)
      SWITCH choice:
        CALL corresponding Hospital method
    END LOOP

END
```

4. Complete C++ Implementation

The full, compiled, and tested source code (hospital.cpp) is listed below.

```
/* *****  
 * Smart Hospital Bed & Patient Monitoring Management System  
 * DSA01 - Object Oriented Programming with C++  
 *  
 * Demonstrates: Encapsulation, Inheritance, Polymorphism (virtual functions),  
 * Constructors (default/parameterized/copy), Destructors, Static Members,  
 * Collections of Objects (vector), Operator Overloading (>, <<),  
 * Control Structures, Menu-Driven Interface.  
 * ***** */  
#include <iostream>  
#include <vector>  
#include <string>  
#include <iomanip>  
#include <limits>  
using namespace std;  
  
/* =====  
 * BASE CLASS : Patient (Encapsulation + abstract base for hierarchy)  
 * ===== */  
class Patient {  
protected:  
    int patientID;  
    string name;  
    int age;  
    string diagnosis;  
  
    static int totalPatients;          // Static member: shared across all patients  
  
public:  
    // Default constructor  
    Patient() : patientID(0), name("Unknown"), age(0), diagnosis("N/A") {  
        totalPatients++;  
    }  
  
    // Parameterized constructor  
    Patient(int id, const string &n, int a, const string &diag)  
        : patientID(id), name(n), age(a), diagnosis(diag) {  
        totalPatients++;  
    }  
  
    // Copy constructor  
    Patient(const Patient &p)  
        : patientID(p.patientID), name(p.name), age(p.age), diagnosis(p.diagnosis) {  
        totalPatients++;  
    }  
  
    // Virtual destructor - essential for polymorphic base classes  
    virtual ~Patient() {  
        totalPatients--;  
    }  
  
    // Pure virtual functions -> run-time polymorphism  
    virtual double dailyCharge() const = 0;  
    virtual double serviceUnitCharge() const = 0;  
    virtual int priorityLevel() const = 0;    // 1 = highest priority  
    virtual string category() const = 0;  
  
    // Accessors  
    int getID() const { return patientID; }  
    string getName() const { return name; }
```

```

int getAge() const { return age; }
string getDiagnosis() const { return diagnosis; }

virtual void display() const {
    cout << left << setw(6) << patientID << setw(16) << name << setw(5) << age
        << setw(12) << category() << setw(22) << diagnosis
        << "Rs." << dailyCharge() << "/day" << endl;
}

static int getTotalPatients() { return totalPatients; }
};
int Patient::totalPatients = 0;

/* -----
 * DERIVED CLASSES : GeneralPatient, ICUPatient, EmergencyPatient
 * Inheritance + overriding -> different billing / monitoring / priority
 * ----- */
class GeneralPatient : public Patient {
public:
    GeneralPatient() : Patient() {}
    GeneralPatient(int id, const string &n, int a, const string &diag)
        : Patient(id, n, a, diag) {}
    GeneralPatient(const GeneralPatient &g) : Patient(g) {}

    double dailyCharge() const override { return 2000.0; }
    double serviceUnitCharge() const override { return 300.0; }
    int priorityLevel() const override { return 3; } // lowest priority
    string category() const override { return "General"; }
};

class ICUPatient : public Patient {
public:
    ICUPatient() : Patient() {}
    ICUPatient(int id, const string &n, int a, const string &diag)
        : Patient(id, n, a, diag) {}
    ICUPatient(const ICUPatient &i) : Patient(i) {}

    double dailyCharge() const override { return 7500.0; }
    double serviceUnitCharge() const override { return 1000.0; }
    int priorityLevel() const override { return 2; } // high, continuous monitoring
    string category() const override { return "ICU"; }
};

class EmergencyPatient : public Patient {
public:
    EmergencyPatient() : Patient() {}
    EmergencyPatient(int id, const string &n, int a, const string &diag)
        : Patient(id, n, a, diag) {}
    EmergencyPatient(const EmergencyPatient &e) : Patient(e) {}

    double dailyCharge() const override { return 5000.0; }
    double serviceUnitCharge() const override { return 750.0; }
    int priorityLevel() const override { return 1; } // highest priority allocation
    string category() const override { return "Emergency"; }
};

/* =====
 * HospitalBed (Encapsulation)
 * ===== */
class HospitalBed {
private:
    int bedNumber;
    string wardType;
    bool occupied;

```

```

    int occupantID;        // -1 if free

public:
    HospitalBed() : bedNumber(0), wardType("General"), occupied(false), occupantID(-1) {}
    HospitalBed(int num, const string &ward)
        : bedNumber(num), wardType(ward), occupied(false), occupantID(-1) {}
    HospitalBed(const HospitalBed &b)
        : bedNumber(b.bedNumber), wardType(b.wardType),
          occupied(b.occupied), occupantID(b.occupantID) {}

    int getBedNumber() const { return bedNumber; }
    string getWardType() const { return wardType; }
    bool isOccupied() const { return occupied; }
    int getOccupantID() const { return occupantID; }

    void occupy(int patientID) { occupied = true; occupantID = patientID; }
    void vacate() { occupied = false; occupantID = -1; }

    void display() const {
        cout << left << setw(6) << bedNumber << setw(12) << wardType
              << setw(12) << (occupied ? "Occupied" : "Available");
        if (occupied) cout << "Patient ID: " << occupantID;
        cout << endl;
    }
};

/* =====
 * Admission (Constructors/Destructor/Operator overloading)
 * ===== */
class Admission {
private:
    int admissionID;
    int patientID;
    string patientName;
    int bedNumber;
    int days;
    int serviceUnits;
    double bedCharge;        // per-day charge captured at admission time
    double serviceCharge;    // per-unit charge captured at admission time
    double bill;
    bool active;

    static int totalAdmissions;    // static: shared hospital-level counter
    static double totalRevenue;    // static: shared hospital-level revenue

public:
    // Default constructor
    Admission()
        : admissionID(0), patientID(0), patientName(""), bedNumber(0),
          days(1), serviceUnits(0), bedCharge(0), serviceCharge(0),
          bill(0), active(false) {}

    // Parameterized constructor
    Admission(int admID, Patient *p, int bedNo)
        : admissionID(admID), patientID(p->getID()), patientName(p->getName()),
          bedNumber(bedNo), days(1), serviceUnits(0),
          bedCharge(p->dailyCharge()), serviceCharge(p->serviceUnitCharge()),
          bill(0), active(true) {
        totalAdmissions++;
    }

    // Copy constructor
    Admission(const Admission &a)
        : admissionID(a.admissionID), patientID(a.patientID), patientName(a.patientName),

```

```

        bedNumber(a.bedNumber), days(a.days), serviceUnits(a.serviceUnits),
        bedCharge(a.bedCharge), serviceCharge(a.serviceCharge),
        bill(a.bill), active(a.active) {
            totalAdmissions++;
        }

// Destructor - finalizes admission record (bed release handled by Hospital)
~Admission() {
    totalAdmissions--;
}

void setDays(int d) { if (d > 0) days = d; }
int getDays() const { return days; }

void updateService(int units) { if (units > 0) serviceUnits += units; }
int getServiceUnits() const { return serviceUnits; }

double calculateBill() {
    bill = (bedCharge * days) + (serviceCharge * serviceUnits);
    return bill;
}

double getBill() const { return bill; }
int getPatientID() const { return patientID; }
string getPatientName() const { return patientName; }
int getBedNumber() const { return bedNumber; }
int getAdmissionID() const { return admissionID; }
bool isActive() const { return active; }

void discharge() {
    calculateBill();
    active = false;
    totalRevenue += bill;
}

// Operator overload #1: compare two admissions by final/current bill
bool operator>(const Admission &other) const {
    return this->bill > other.bill;
}

// Operator overload #2: format admission record for report output
friend ostream& operator<<(ostream &os, const Admission &a) {
    os << left << setw(6) << a.admissionID << setw(6) << a.patientID
        << setw(18) << a.patientName << setw(6) << a.bedNumber
        << setw(6) << a.days << setw(10) << a.serviceUnits
        << "Rs." << fixed << setprecision(2) << a.bill
        << (a.active ? " [Active]" : " [Discharged]");
    return os;
}

static int getTotalAdmissions() { return totalAdmissions; }
static double getTotalRevenue() { return totalRevenue; }
};

int Admission::totalAdmissions = 0;
double Admission::totalRevenue = 0.0;

/* =====
 * Hospital (Aggregation: manages collections, allocation, reports, menu)
 * ===== */
class Hospital {
private:
    vector<Patient*> patients;
    vector<HospitalBed> beds;
    vector<Admission> admissions;

```

```

    int nextAdmissionID;

public:
    Hospital(int totalBeds) : nextAdmissionID(1) {
        // Allocate a fixed number of beds across ward types
        for (int i = 1; i <= totalBeds; i++) {
            string ward = (i % 3 == 0) ? "ICU" : (i % 3 == 1) ? "General" : "Emergency";
            beds.push_back(HospitalBed(i, ward));
        }
    }

    ~Hospital() {
        for (Patient* p : patients) delete p;    // release dynamically allocated patients
        patients.clear();
    }

    /* ---- Input helpers ---- */
    static int readInt(const string &prompt) {
        int val;
        while (true) {
            cout << prompt;
            if (cin >> val) { cin.ignore(numeric_limits<streamsize>::max(), '\n'); return val; }
            cout << "Invalid input. Please enter a number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }

    static string readLine(const string &prompt) {
        string val;
        cout << prompt;
        getline(cin, val);
        return val;
    }

    /* 1. Patient Registration */
    void registerPatient() {
        int id = readInt("Enter Patient ID: ");
        if (findPatient(id) != nullptr) {
            cout << "Error: Patient ID already exists.\n";
            return;
        }
        string name = readLine("Enter Name: ");
        int age = readInt("Enter Age: ");
        string diag = readLine("Enter Diagnosis: ");
        cout << "Select Category -> 1.General 2.ICU 3.Emergency : ";
        int cat = readInt("");

        Patient *p = nullptr;
        switch (cat) {
            case 1: p = new GeneralPatient(id, name, age, diag); break;
            case 2: p = new ICUPatient(id, name, age, diag); break;
            case 3: p = new EmergencyPatient(id, name, age, diag); break;
            default:
                cout << "Invalid category. Registration cancelled.\n";
                return;
        }
        patients.push_back(p);
        cout << "Patient registered successfully. Total registered patients (static count): "
              << Patient::getTotalPatients() << endl;
    }

    /* 2. Display All Patients */
    void displayPatients() const {
        if (patients.empty()) { cout << "No patients registered.\n"; return; }
    }

```



```

        cout << left << setw(6) << "ID" << setw(16) << "Name" << setw(5) << "Age"
            << setw(12) << "Category" << setw(22) << "Diagnosis" << "Daily Charge\n";
        cout << string(70, '-') << endl;
        for (Patient* p : patients) p->display();    // run-time polymorphism
    }

    /* 3. Display All Beds */
    void displayBeds() const {
        cout << left << setw(6) << "Bed#" << setw(12) << "Ward" << setw(12) << "Status" <<
"Occupant\n";
        cout << string(50, '-') << endl;
        for (const auto &b : beds) b.display();
    }

    /* 4. Admit Patient / Allocate Bed (priority-based allocation) */
    void admitPatient() {
        int pid = readInt("Enter Patient ID to admit: ");
        Patient *p = findPatient(pid);
        if (!p) { cout << "Error: Patient not found.\n"; return; }
        if (findActiveAdmission(pid) != nullptr) {
            cout << "Error: Patient already admitted.\n"; return;
        }

        // Allocate a matching-ward free bed first; ICU/Emergency (priority<=2)
        // may fall back to any free bed if their ward type is full.
        HospitalBed *chosen = nullptr;
        for (auto &b : beds)
            if (!b.isOccupied() && b.getWardType() == p->category()) { chosen = &b; break; }
        if (!chosen && p->priorityLevel() <= 2) {
            for (auto &b : beds) if (!b.isOccupied()) { chosen = &b; break; }
        }
        if (!chosen) {
            cout << "No available bed for this patient category at the moment.\n";
            return;
        }

        chosen->occupy(pid);
        Admission adm(nextAdmissionID++, p, chosen->getBedNumber());
        admissions.push_back(adm);
        cout << "Bed " << chosen->getBedNumber() << " (" << chosen->getWardType()
            << ") allocated. Admission ID: " << adm.getAdmissionID() << endl;
    }

    /* 5. Update Monitoring or Service Usage */
    void updateMonitoring() {
        int pid = readInt("Enter Patient ID: ");
        Admission *adm = findActiveAdmission(pid);
        if (!adm) { cout << "Error: No active admission for this patient.\n"; return; }
        int units = readInt("Enter number of monitoring/service units to add: ");
        adm->updateService(units);
        cout << "Service units updated. Total units so far: " << adm->getServiceUnits() << endl;
    }

    /* 6. Calculate Current Bill */
    void calculateBill() {
        int pid = readInt("Enter Patient ID: ");
        Admission *adm = findActiveAdmission(pid);
        if (!adm) { cout << "Error: No active admission for this patient.\n"; return; }
        int days = readInt("Enter number of days stayed so far: ");
        adm->setDays(days);
        double bill = adm->calculateBill();
        cout << "Current bill for Patient " << pid << ": Rs." << fixed << setprecision(2) << bill <<
endl;
    }
}

```

```

/* 7. Patient Discharge */
void dischargePatient() {
    int pid = readInt("Enter Patient ID to discharge: ");
    Admission *adm = findActiveAdmission(pid);
    if (!adm) { cout << "Error: No active admission for this patient.\n"; return; }
    adm->discharge();
    for (auto &b : beds)
        if (b.getBedNumber() == adm->getBedNumber()) { b.vacate(); break; }
    cout << "Patient " << pid << " discharged. Final Bill: Rs."
        << fixed << setprecision(2) << adm->getBill()
        << ". Bed " << adm->getBedNumber() << " released.\n";
}

/* 8. Display Admission Records (uses operator<<) */
void displayAdmissions() const {
    if (admissions.empty()) { cout << "No admission records.\n"; return; }
    cout << left << setw(6) << "AdmID" << setw(6) << "PatID" << setw(18) << "Name"
        << setw(6) << "Bed" << setw(6) << "Days" << setw(10) << "Units" << "Bill\n";
    cout << string(70, '-') << endl;
    for (const auto &a : admissions) cout << a << endl;    // operator<< overload
}

/* 9. Compare Two Admissions (uses operator>) */
void compareAdmissions() {
    int id1 = readInt("Enter first Admission ID: ");
    int id2 = readInt("Enter second Admission ID: ");
    Admission *a1 = findAdmissionByID(id1);
    Admission *a2 = findAdmissionByID(id2);
    if (!a1 || !a2) { cout << "Error: One or both admission IDs not found.\n"; return; }
    a1->calculateBill();
    a2->calculateBill();
    cout << "Admission " << id1 << " -> " << *a1 << endl;
    cout << "Admission " << id2 << " -> " << *a2 << endl;
    if (*a1 > *a2) cout << "Admission " << id1 << " has a higher bill.\n";
    else if (*a2 > *a1) cout << "Admission " << id2 << " has a higher bill.\n";
    else cout << "Both admissions have equal bills.\n";
}

/* 10. Bed Utilization Report */
void utilizationReport() const {
    int total = (int)beds.size();
    int occupied = 0;
    for (const auto &b : beds) if (b.isOccupied()) occupied++;
    int available = total - occupied;
    double utilization = total > 0 ? (occupied * 100.0 / total) : 0.0;

    int activeAdmissions = 0;
    for (const auto &a : admissions) if (a.isActive()) activeAdmissions++;

    cout << "\n----- Bed Utilization Report ----- \n";
    cout << "Total Beds          : " << total << endl;
    cout << "Occupied Beds         : " << occupied << endl;
    cout << "Available Beds        : " << available << endl;
    cout << "Utilization %         : " << fixed << setprecision(1) << utilization << "%\n";
    cout << "Active Admissions     : " << activeAdmissions << endl;
    cout << "Total Admissions (all): " << Admission::getTotalAdmissions() << endl;
    cout << "Total Revenue Booked  : Rs." << fixed << setprecision(2)
        << Admission::getTotalRevenue() << endl;
    cout << "----- \n";
}

/* ---- Search helpers ---- */
Patient* findPatient(int id) const {

```

```

        for (Patient* p : patients) if (p->getID() == id) return p;
        return nullptr;
    }
    Admission* findActiveAdmission(int patientID) {
        for (auto &a : admissions) if (a.getPatientID() == patientID && a.isActive()) return &a;
        return nullptr;
    }
    Admission* findAdmissionByID(int admID) {
        for (auto &a : admissions) if (a.getAdmissionID() == admID) return &a;
        return nullptr;
    }
};

/* =====
 * MAIN : Menu-driven control flow
 * ===== */
void printMenu() {
    cout << "\n===== SMART HOSPITAL BED & PATIENT MONITORING =====\n";
    cout << "1. Register Patient\n";
    cout << "2. Display All Patients\n";
    cout << "3. Display All Beds\n";
    cout << "4. Admit Patient / Allocate Bed\n";
    cout << "5. Update Monitoring or Service Usage\n";
    cout << "6. Calculate Current Bill\n";
    cout << "7. Discharge Patient\n";
    cout << "8. Display Admission Records\n";
    cout << "9. Compare Two Admissions\n";
    cout << "10. Generate Bed Utilization Report\n";
    cout << "0. Exit\n";
    cout << "=====\n";
}

int main() {
    Hospital hospital(9); // fixed number of hospital beds
    int choice;

    do {
        printMenu();
        choice = Hospital::readInt("Enter your choice: ");
        switch (choice) {
            case 1: hospital.registerPatient(); break;
            case 2: hospital.displayPatients(); break;
            case 3: hospital.displayBeds(); break;
            case 4: hospital.admitPatient(); break;
            case 5: hospital.updateMonitoring(); break;
            case 6: hospital.calculateBill(); break;
            case 7: hospital.dischargePatient(); break;
            case 8: hospital.displayAdmissions(); break;
            case 9: hospital.compareAdmissions(); break;
            case 10: hospital.utilizationReport(); break;
            case 0: cout << "Exiting system. Goodbye!\n"; break;
            default: cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 0);

    return 0;
}

```

5. Sample Input / Output (Execution Transcript)

The program was compiled with g++ (C++17, -Wall, zero warnings) and exercised through a full session covering all ten menu options. An abridged, readable transcript is shown below (the menu banner is shown once for brevity).

```
===== SMART HOSPITAL BED & PATIENT MONITORING =====
1. Register Patient    2. Display All Patients    3. Display All Beds
4. Admit Patient       5. Update Monitoring      6. Calculate Bill
7. Discharge Patient   8. Display Admissions    9. Compare Admissions
10. Utilization Report 0. Exit
=====

>> Registering 3 patients (IDs 101 General, 102 ICU, 103 Emergency)
Patient registered successfully. Total registered patients (static count): 1
Patient registered successfully. Total registered patients (static count): 2
Patient registered successfully. Total registered patients (static count): 3

>> Option 2: Display All Patients
ID   Name      Age  Category  Diagnosis              Daily Charge
-----
101  Ravi Kumar  45   General   Fracture - left leg    Rs.2000/day
102  Meena Iyer  60   ICU       Post-cardiac surgery   Rs.7500/day
103  Arjun Dev   30   Emergency Road accident trauma   Rs.5000/day

>> Option 4: Admit each patient (bed allocated by matching ward / priority)
Bed 1 (General) allocated. Admission ID: 1
Bed 3 (ICU) allocated. Admission ID: 2
Bed 2 (Emergency) allocated. Admission ID: 3

>> Option 3: Display All Beds (after admission)
Bed#  Ward      Status    Occupant
-----
1     General   Occupied  Patient ID: 101
2     Emergency Occupied  Patient ID: 103
3     ICU       Occupied  Patient ID: 102
4     General   Available
5     Emergency Available
6     ICU       Available
7     General   Available
8     Emergency Available
9     ICU       Available

>> Option 5: Update Monitoring (Patient 102, +4 service units)
Service units updated. Total units so far: 4

>> Option 6: Calculate Current Bill (Patient 101, 2 days)
Current bill for Patient 101: Rs.4000.00

>> Option 7: Discharge Patient 101
Patient 101 discharged. Final Bill: Rs.4000.00. Bed 1 released.

>> Option 9: Compare Two Admissions (Admission 2 vs Admission 3)
Admission 2 -> 2 102 Meena Iyer 3 1 4 Rs.11500.00 [Active]
Admission 3 -> 3 103 Arjun Dev 2 1 0 Rs.5000.00 [Active]
Admission 2 has a higher bill.

>> Option 10: Generate Bed Utilization Report
----- Bed Utilization Report -----
Total Beds      : 9
Occupied Beds    : 2
Available Beds   : 7
Utilization %    : 22.2%
```

Active Admissions : 2
Total Admissions (all): 3
Total Revenue Booked : Rs.4000.00

>> Option 8: Display Admission Records

AdmID	PatID	Name	Bed	Days	Units	Bill
1	101	Ravi Kumar	1	2	0	Rs.4000.00 [Discharged]
2	102	Meena Iyer	3	1	4	Rs.11500.00 [Active]
3	103	Arjun Dev	2	1	0	Rs.5000.00 [Active]

>> Option 0: Exit
Exiting system. Goodbye!

6. Explanation of Inheritance and Polymorphism

Patient is declared as an abstract base class with four pure virtual functions: `dailyCharge()`, `serviceUnitCharge()`, `priorityLevel()`, and `category()`. `GeneralPatient`, `ICUPatient`, and `EmergencyPatient` publicly inherit from `Patient` and override each of these functions with category-specific values.

The `Hospital` class stores patients as a `vector<Patient*>` — a collection of base-class pointers. When `displayPatients()` calls `p->display()` (which internally calls the virtual charge/category functions), or when `admitPatient()` calls `p->dailyCharge()` or `p->priorityLevel()`, the correct derived-class version runs automatically based on the object's actual run-time type, not its declared pointer type. This is classic run-time polymorphism through virtual dispatch, and it is what lets one uniform loop over all patients produce different billing, monitoring, and bed-priority behaviour for General, ICU, and Emergency patients without any type-checking (if/else on category) in the calling code.

The `Patient` destructor is declared virtual, which is essential: because patients are deleted through a base-class pointer (delete `p` in `Hospital`'s destructor), a non-virtual destructor would only run `Patient`'s own cleanup and skip the derived class's part, causing incomplete destruction. Declaring it virtual guarantees the most-derived destructor runs first, followed by the base — correct, safe cleanup regardless of which concrete patient type is stored.

7. Reflection

Why Inheritance and Polymorphism Suit Different Patient Categories

General, ICU, and Emergency patients share the same core identity (ID, name, age, diagnosis) but differ in how they are billed, monitored, and prioritized for bed allocation. Modelling this as an inheritance hierarchy avoids duplicating the shared fields in three separate classes, while polymorphism (virtual functions) lets a single `Hospital` class treat every patient uniformly through a `Patient*` — the correct category-specific behaviour is selected automatically at run time. This keeps the system open to new patient categories (e.g., a future `MaternityPatient`) without modifying existing `Hospital` logic — an example of the open/closed principle in practice.

Role of Constructors and Destructors in Object/Bed Lifecycles

Parameterized constructors create a fully-initialized `Patient` or `Admission` in one step, preventing half-constructed objects. Copy constructors ensure that when an `Admission` or `Patient` is duplicated (e.g., passed by value or stored in a container that reallocates), each copy owns its own independent data. Destructors close the lifecycle: `Admission`'s destructor decrements the static `totalAdmissions` counter so hospital-level statistics stay accurate even as records go out of scope, and `Hospital`'s destructor frees every dynamically-allocated `Patient*` to avoid memory leaks. Bed release itself is handled explicitly in `dischargePatient()` (rather than only in a destructor) so that a bed becomes available immediately when a patient is discharged, not only when the program ends.

Why the Chosen Operator Overloads Are Meaningful

`operator>` on `Admission` compares two admissions by their computed bill, which directly supports the "Compare Two Admissions" requirement (option 9) with natural, readable syntax (`a1 > a2`) instead of a separate `compareBills()` function. `operator<<` on `Admission` overloads stream insertion to format a full admission record (ID, patient, bed, days, units, bill, status) consistently everywhere it is printed — in the admission list and in the comparison output — so the report format is defined once and reused, reducing duplication and formatting drift.

SDG Relevance

SDG 3 (Good Health and Well-being): automated, priority-aware bed allocation reduces delays in getting Emergency and ICU patients into an appropriate bed, and consistent monitoring/billing tracking reduces administrative errors that could otherwise affect patient care.

SDG 9 (Industry, Innovation and Infrastructure): the system demonstrates how a modest software application can modernize hospital infrastructure — replacing manual bed registers and paper billing with a structured, auditable digital workflow that scales more easily than manual tracking as bed count or patient volume grows.

Implementation Challenge and Resolution

One challenge was deciding how to release a bed correctly when a patient with dynamically-selected polymorphic behaviour is discharged, without accidentally deleting the Patient object itself (a patient's registration record should persist for history even after discharge). This was solved by separating concerns: Admission only stores the patientID and bedNumber (not object ownership), dischargePatient() finalizes the bill and marks the admission inactive, and the Hospital explicitly calls HospitalBed::vacate() on the matching bed. Patient objects are only freed once, in Hospital's own destructor, which also confirmed the importance of using a virtual destructor on Patient to avoid partial (sliced) destruction through the base pointer.

8. Deliverables Checklist

- Pseudocode — Section 3
- Complete C++ implementation — Section 4 (compiles cleanly with g++ -std=c++17 -Wall, zero warnings)
- Sample input/output — Section 5
- Explanation of inheritance and polymorphism — Section 6
- Reflection and SDG mapping — Section 7
- GitHub upload of source code — to be completed by student after submission